

- Alter whether an existing constraint named *symbol* is enforced:

```
ALTER TABLE tbl_name
  ALTER CONSTRAINT symbol [NOT] ENFORCED;
```

Only **CHECK** constraints can be altered to be unenforced. All other constraint types are always enforced.

The SQL standard specifies that all types of constraints (primary key, unique index, foreign key, check) belong to the same namespace. In MySQL, each constraint type has its own namespace per schema. Consequently, names for each type of constraint must be unique per schema, but constraints of different types can have the same name. When multiple constraints have the same name, **DROP CONSTRAINT** and **ADD CONSTRAINT** are ambiguous and an error occurs. In such cases, constraint-specific syntax must be used to modify the constraint. For example, use **DROP PRIMARY KEY** or **DROP FOREIGN KEY** to drop a primary key or foreign key.

If a table alteration causes a violation of an enforced **CHECK** constraint, an error occurs and the table is not modified. Examples of operations for which an error occurs:

- Attempts to add the **AUTO_INCREMENT** attribute to a column that is used in a **CHECK** constraint.
- Attempts to add an enforced **CHECK** constraint or enforce a nonenforced **CHECK** constraint for which existing rows violate the constraint condition.
- Attempts to modify, rename, or drop a column that is used in a **CHECK** constraint, unless that constraint is also dropped in the same statement. Exception: If a **CHECK** constraint refers only to a single column, dropping the column automatically drops the constraint.

ALTER TABLE *tbl_name* RENAME *new_tbl_name* changes internally generated and user-defined **CHECK** constraint names that begin with the string "*tbl_name*_chk_" to reflect the new table name. MySQL interprets **CHECK** constraint names that begin with the string "*tbl_name*_chk_" as internally generated names.

Changing the Character Set

To change the table default character set and all character columns (**CHAR**, **VARCHAR**, **TEXT**) to a new character set, use a statement like this:

```
ALTER TABLE tbl_name CONVERT TO CHARACTER SET charset_name;
```

The statement also changes the collation of all character columns. If you specify no **COLLATE** clause to indicate which collation to use, the statement uses default collation for the character set. If this collation is inappropriate for the intended table use (for example, if it would change from a case-sensitive collation to a case-insensitive collation), specify a collation explicitly.

For a column that has a data type of **VARCHAR** or one of the **TEXT** types, **CONVERT TO CHARACTER SET** changes the data type as necessary to ensure that the new column is long enough to store as many characters as the original column. For example, a **TEXT** column has two length bytes, which store the byte-length of values in the column, up to a maximum of 65,535. For a **latin1 TEXT** column, each character requires a single byte, so the column can store up to 65,535 characters. If the column is converted to **utf8**, each character might require up to three bytes, for a maximum possible length of $3 \times 65,535 = 196,605$ bytes. That length does not fit in a **TEXT** column's length bytes, so MySQL converts the data type to **MEDIUMTEXT**, which is the smallest string type for which the length bytes can record a value of 196,605. Similarly, a **VARCHAR** column might be converted to **MEDIUMTEXT**.

To avoid data type changes of the type just described, do not use **CONVERT TO CHARACTER SET**. Instead, use **MODIFY** to change individual columns. For example:

```
ALTER TABLE t MODIFY latin1_text_col TEXT CHARACTER SET utf8;
ALTER TABLE t MODIFY latin1_varchar_col VARCHAR(M) CHARACTER SET utf8;
```